

UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA



CURSO:

Algoritmos y Estructura de Datos 1
(BMA09-Q)

TEMA DEL INFORME:

Divide and Conquer

INTEGRANTES:

HUATUCO ROJAS, Sebastian Matias
ALEGRE BUSTAMANTE, Max André
FLORES APAZA, Joel Josue
(Grupo 1)

DOCENTE:

SELIS VASQUEZ, Luis Antonio

FECHA DE ENTREGA:

05/10/2026

ÍNDICE

● 1. Objetivos	2
● 2. Introducción	2
● 3. Concepto de Divide and Conquer	3
○ 3.1. Definición y Filosofía	3
○ 3.2. Características Principales	3
○ 3.3. Marco de Medición: Complejidad Computacional y Notación Big-O	3
■ 3.3.1 Sustento y Análisis Matemático de las Complejidades	4
○ 3.4. Diferencia Técnica: Divide and Conquer vs. Fuerza Bruta	4
○ 3.5. Ejemplo Práctico: La Analogía del Diccionario	5
● 4. Estrategia Divide and Conquer	5
○ 4.1. El “Ciclo de Vida” del Paradigma	5
● 5. División del problema	6
● 6. Resolución de subproblemas	7
● 7. Combinación de soluciones	7
● 8. Recurrencias	7
● 9. Teorema Maestro	8
● 10. Algoritmos representativos	9
○ 10.1. Búsqueda Binaria (Binary Search)	9
■ 10.1.1. Fase de División (Partición del Dominio)	9
■ 10.1.2. Fase de Vencimiento (Procesamiento y Caso Base)	9
■ 10.1.3. Fase de Combinación (Integración)	9
■ 10.1.4. Impacto en la Eficiencia	9
○ 10.2. Ordenamiento por Fusión (Merge Sort)	10
■ 10.2.1. Aplicación del paradigma Divide y Vencerás	10
■ 10.2.2. Formulación y análisis de complejidad temporal	10
■ 10.2.3. Resolución mediante el Teorema Maestro	10
■ 10.2.4. Evaluación de propiedades operativas	11
○ 10.3. Ordenamiento Rápido (Quick Sort)	11
■ 10.3.1. Aplicación del paradigma Divide y Vencerás	11
■ 10.3.2. Formulación y análisis de complejidad temporal	11
■ 10.3.3. Evaluación de propiedades operativas	12
○ 10.4. Tabla Comparativa General	12
● 11. Conclusiones	13
● 12. Bibliografía	14

1. Objetivos

El presente informe tiene como principal objetivo comprender, analizar e implementar el paradigma de diseño de algoritmos Divide y Vencerás (*Divide and Conquer*), mediante la formulación teórica de sus componentes, el análisis matemático de su complejidad computacional y el desarrollo de programas funcionales en lenguaje C, para evaluar su eficiencia y aplicabilidad en la resolución de problemas informáticos. Además de profundizar en temas específicos como:

- **Estudiar la estructura del paradigma:** Explicar las tres etapas fundamentales del enfoque (Dividir, Vencer y Combinar) y sus características principales en el diseño de algoritmos.
- **Modelar el rendimiento mediante matemáticas:** Formular relaciones de recurrencia y aplicar el Teorema Maestro para determinar de forma rigurosa la complejidad temporal y espacial de los algoritmos estudiados.
- **Implementar soluciones en lenguaje C:** Desarrollar el código fuente de algoritmos representativos (como Merge Sort, Quick Sort y Búsqueda Binaria), garantizando un código limpio, estructurado y verificado mediante pruebas de ejecución.
- **Evaluar y comparar algoritmos:** Analizar las ventajas, desventajas, rendimiento e idoneidad de cada algoritmo estudiado según el contexto del problema y sus aplicaciones en el mundo real.

2. Introducción

En el ámbito de la ciencia de la computación y la ingeniería de software, el diseño eficiente de algoritmos es un pilar fundamental para la resolución de problemas complejos. A medida que el volumen de datos procesados por los sistemas modernos crece exponencialmente, la elección de una estrategia algorítmica adecuada determina la viabilidad y el rendimiento de una solución informática. Entre los diversos paradigmas de diseño, Divide y Vencerás (*Divide and Conquer*) destaca como uno de los enfoques más potentes, elegantes y utilizados para optimizar el tiempo de ejecución y el uso de recursos.

El presente trabajo de investigación analiza de manera teórica y práctica la estrategia de Divide y Vencerás, desglosando su ciclo de vida característico basado en tres etapas esenciales: la división del problema en subproblemas más pequeños del mismo tipo, la resolución o conquista de dichos subproblemas mediante recursión, y la combinación de las soluciones parciales para obtener la respuesta al problema original. Asimismo, se aborda el modelado matemático necesario para evaluar su eficiencia a través de la formulación de relaciones de recurrencia y la resolución de las mismas mediante el Teorema Maestro.

Para validar los conceptos teóricos expuestos, se presentan e implementan en lenguaje C algoritmos emblemáticos de este paradigma, tales como Merge Sort, Quick Sort y la Búsqueda Binaria. A través de estas implementaciones y sus correspondientes pruebas de ejecución, se ofrece un análisis comparativo de su complejidad temporal y espacial, sus ventajas y desventajas, así como sus principales aplicaciones en escenarios reales del desarrollo de software.

3. Concepto de Divide and Conquer

3.1. Definición y Filosofía

El paradigma Divide y Vencerás (*Divide and Conquer*) es una estrategia de diseño de algoritmos basada en el principio de resolución recursiva de problemas. Su filosofía central radica en la premisa de que un problema complejo y de gran tamaño resulta más fácil de resolver si se descompone de forma sistemática en subproblemas más pequeños de la misma naturaleza.

Una vez que estos subproblemas se reducen a un tamaño lo suficientemente simple (denominado caso base), su solución es inmediata. Posteriormente, las soluciones de los subproblemas se combinan para construir la solución completa del problema original.

3.2. Características Principales

- **Descomposición Recursiva:** El algoritmo se invoca a sí mismo sobre instancias del problema con un volumen de datos cada vez menor, descendiendo por una estructura jerárquica hasta alcanzar el caso base.
- **Independencia de Subproblemas:** Cada subproblema generado es autónomo y no depende de la resolución de los otros subproblemas en la misma etapa, lo que garantiza que las llamadas recursivas no generen interferencias ni dependencias cruzadas.
- **Reducción sustancial de la complejidad computacional:** Al evitar procesar el conjunto total de datos de forma lineal, la estrategia logra transformar algoritmos de complejidad cuadrática en algoritmos de complejidad cuasilineal o logarítmica.
- **Aptitud para el Procesamiento en Paralelo:** Debido a la independencia entre los subproblemas, este paradigma se presta de manera natural para la ejecución concurrente en arquitecturas de procesadores multinúcleo.

3.3. Marco de Medición: Complejidad Computacional y Notación Big-O

Para evaluar la eficiencia de un algoritmo no se utiliza el tiempo medido en segundos con un cronómetro, ya que este varía según la potencia del procesador o la memoria RAM de la computadora. En su lugar, se utiliza la Complejidad Computacional mediante la Notación Asintótica Big-O (O), la cual mide cómo escala el uso de recursos a medida que el tamaño de la entrada de datos (n) tiende al infinito.

Complejidad Temporal $O(f(n))$:

- **¿Qué representa?** Representa el número total de operaciones o instrucciones básicas (comparaciones, asignaciones, multiplicaciones) que ejecuta el algoritmo para procesar n elementos.
- **Objetivo:** Garantizar que el programa resuelva el problema en el menor número de pasos posible.

Complejidad Espacial $O(g(n))$:

- **¿Qué representa?** Representa la memoria RAM adicional que requiere el algoritmo para ejecutarse, más allá del espacio ocupado por la entrada original de datos.
- **En Divide y Vencerás:** Incluye el espacio consumido en la memoria por la pila de llamadas recursivas

(*call stack*), la cual almacena el estado de las funciones pendientes mientras se resuelven los subproblemas.

3.3.1 Sustento y Análisis Matemático de las Complejidades

A continuación se demuestra matemáticamente qué representa cada complejidad usando el modelo de división por la mitad:

A. Demostración de la Complejidad Logarítmica: $O(\log n)$

En un algoritmo de Divide y Vencerás, en cada paso el problema de tamaño n se reduce a la mitad. El proceso termina cuando el problema se reduce a tamaño 1 (el caso base). Si llamamos k al número total de divisiones (pasos) realizadas:

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \log_2(n) = k$$

Sustento Matemático: El número de pasos k que ejecuta el algoritmo es exactamente $\log_2(n)$. En notación asintótica se omite la base del logaritmo, resultando en una complejidad temporal de $O(\log n)$.

B. Sustento de la Complejidad Cuasilineal: $O(n \log n)$

Esta complejidad surge cuando un algoritmo divide el problema en un árbol de recursión de profundidad $(\log_2 n)$ (como se demostró arriba), pero en cada nivel del árbol debe procesar o combinar los n elementos completos.

$$\text{Trabajo Total} = (\text{Profundidad del Árbol}) * (\text{Trabajo por Nivel}) = \log_2(n) * n = n \log_2(n)$$

Sustento Matemático: Aplicando las reglas de la notación Big-O, la constante de la base logarítmica se omite, resultando en una complejidad temporal de $O(n \log n)$. Es el límite matemático óptimo para algoritmos de ordenamiento basados en comparaciones.

C. Sustento de la Complejidad Espacial en Recursión

Cuando una función recursiva se divide en $\log_2(n)$ niveles hasta alcanzar el caso base, la memoria RAM debe mantener activa cada llamada intermedia en la pila (*stack frame*).

$$\text{Memoria Espacial} = (\text{Profundidad Máxima de Pila}) * (\text{Memoria por Llamada})$$

- **Búsqueda Binaria:** La profundidad de la pila es $\log_2(n)$ y cada llamada usa memoria constante $O(1)$. Por tanto, su complejidad espacial recursiva es $O(\log n)$.
- **Merge Sort:** Requiere un arreglo auxiliar para la fase de combinación de n elementos más la pila recursiva de $\log_2(n)$. El término dominante es n , resultando en una complejidad espacial de $O(n)$.

3.4. Diferencia Técnica: Divide and Conquer vs. Fuerza Bruta

La principal distinción entre estos enfoques radica en cómo abordan la escala de los datos y la estructura del flujo de control:

Criterio	Fuerza Bruta	Divide and Conquer (Recursivo)
Estrategia	Prueba todas las combinaciones posibles de forma exhaustiva.	Divide el dominio de entrada en partes proporcionales y resuelve recursivamente.
Complejidad Temporal Típica	Alta: $O(n^2)$, $O(2^n)$ o $O(n!)$.	Optimizada: $O(\log n)$ o $O(n \log n)$.
Uso de Memoria	Generalmente $O(1)$ (no requiere pila).	$O(\log n)$ o $O(n)$ debido a la pila de llamadas (<i>call stack</i>).
Escalabilidad	Deficiente ante grandes volúmenes de datos.	Alta; ideal para grandes volúmenes de datos.

Tabla 1: Comparativa entre Divide y Vencerás vs. Fuerza Bruta.

3.5. Ejemplo Práctico: La Analogía del Diccionario

Un ejemplo cotidiano para ilustrar la diferencia entre el paradigma tradicional de Fuerza Bruta y la estrategia de Divide y Vencerás es la búsqueda de un apellido en un diccionario impreso:

- **Enfoque por Fuerza Bruta (Búsqueda Lineal):** Consiste en abrir la primera página y revisar registro por registro, página por página, hasta encontrar la palabra. Si el diccionario tiene $n = 1000$ páginas, en el peor de los casos se deberán revisar 1000 páginas. Su complejidad computacional es $O(n)$.
- **Enfoque Divide and Conquer (Búsqueda Binaria):** Consiste en abrir el diccionario exactamente por la mitad. Se compara la palabra buscada con el contenido actual:
 - *División:* El problema de 1000 páginas se divide en dos bloques de 500 páginas.
 - *Vencer (Descarte):* Si el apellido alfabéticamente se encuentra antes de la página actual, se descarta por completo la mitad derecha y se conserva únicamente la mitad izquierda (un subproblema de tamaño $n / 2$).
 - *Resultado:* Al repetir este proceso reduciendo el problema a la mitad en cada paso, un diccionario de 1000 páginas se resuelve en un máximo de 10 pasos ($\log_2(1000)$ es aproximadamente 10). Su complejidad computacional se reduce drásticamente a $O(\log n)$.

4. Estrategia Divide and Conquer

4.1. El “Ciclo de Vida” del Paradigma

La estrategia de Divide y Vencerás se basa en un ciclo de tres pasos estructurados para resolver cualquier problema de forma eficiente:

1. Paso 1: Dividir (*Divide*)

En este primer paso, se toma el problema inicial de tamaño n y se parte en dos o más subproblemas más

pequeños.

Idea clave: Los subproblemas generados deben ser de la misma naturaleza que el problema original, pero procesando una cantidad menor de datos.

Ejemplo simple: Si tienes una lista de 100 números para ordenar, el paso de dividir consiste en cortarla por la mitad para obtener dos listas de 50 números.

2. Paso 2: Vencer (*Conquer*)

En este segundo paso se resuelven los subproblemas. El algoritmo toma una decisión según el tamaño de los datos:

- *Si el subproblema es grande:* Se aplica la recursividad, invocando nuevamente al algoritmo para dividirlo en partes aún más pequeñas.
- *Si el subproblema es trivial (Caso Base):* Se resuelve directamente, sin realizar más divisiones.

Caso Base: Es la condición de parada del algoritmo. Corresponde al problema en su forma más simple posible (por ejemplo, una lista de un solo elemento ya está ordenada por definición). Sin un caso base claro, la función recursiva se ejecutaría indefinidamente hasta desbordar la pila de memoria.

3. Paso 3: Combinar (*Combine*)

Una vez resueltos todos los subproblemas, el algoritmo reúne las soluciones parciales para construir la respuesta completa al problema original.

Nota importante: En algunos algoritmos esta etapa requiere un esfuerzo de cómputo significativo (como en Merge Sort, donde es necesario entrelazar los elementos de forma ordenada), mientras que en otros es casi inmediata (como en Quick Sort).

5. División del problema

La fase de división consiste en descomponer la instancia original del problema de tamaño n en un conjunto de a subproblemas de tamaño menor (n / b), manteniendo la misma naturaleza algorítmica del problema raíz.

- **Condición de homogeneidad:** Los subproblemas generados deben ser instancias más pequeñas del mismo tipo de problema. Por ejemplo, en algoritmos de ordenamiento como Merge Sort, dividir una lista desordenada produce dos sublistas que también requieren ser ordenadas.
- **Criterios de partición:**
 - *Partición balanceada:* La entrada se fragmenta en fracciones simétricas (generalmente mitades, como en Merge Sort o Búsqueda Binaria). Maximiza la eficiencia y minimiza la profundidad del árbol recursivo a $O(\log n)$.
 - *Partición basada en pivote/estado:* El punto de corte no siempre es el centro geométrico; depende de un elemento (como el pivote en Quick Sort), lo que puede generar particiones asimétricas según la distribución de los datos.
- **Costo computacional de la división:** El tiempo empleado en partir el problema, denominado $D(n)$, debe ser significativamente menor que resolver el problema completo directamente; típicamente es constante $O(1)$ (calcular índices medios) o lineal $O(n)$ (reorganizar elementos según un pivote).

6. Resolución de subproblemas

La fase de conquista aborda la solución de los subproblemas mediante dos mecanismos fundamentales: la recursión y el caso base.

- **Caso base (condición de parada):** Es el umbral donde el subproblema alcanza un tamaño elemental o indivisible ($n \leq n_0$, comúnmente $n = 1$ o $n = 0$) cuya solución es directa o trivial. En este punto no se realizan más llamadas recursivas:
 - En ordenamiento, un arreglo de 1 elemento ya está ordenado por definición ($O(1)$).
 - En búsqueda, un rango donde el índice inicial supera al final indica inmediatamente la ausencia del elemento.
- **Llamadas recursivas (Paso inductivo):** Si el subproblema excede el caso base, se invoca nuevamente la misma función sobre cada fragmento. Cada llamada opera sobre su propio marco en la pila de ejecución (*call stack*), garantizando que los subproblemas se resuelvan de manera independiente y sin solapamiento (a diferencia de la programación dinámica).
- **Paralelismo intrínseco:** Dado que los subproblemas resultantes son típicamente disjuntos e independientes entre sí, esta etapa permite ejecución concurrente o distribuida en arquitecturas multinúcleo.

7. Combinación de soluciones

La fase de combinación toma las respuestas parciales devueltas por los subproblemas ya resueltos y sintetiza la respuesta al problema original de tamaño n .

- **Naturaleza de la recombinación:** Es frecuentemente donde radica el mayor costo algorítmico del paradigma. Si la división fue trivial, la combinación suele concentrar la lógica pesada:
 - En Merge Sort, la división solo calcula el punto medio ($O(1)$), pero la combinación (*merge*) entrelaza dos listas ordenadas comparando sus elementos cabeza en tiempo lineal $O(n)$.
 - En Quick Sort, el trabajo pesado ocurre en la partición ($O(n)$), por lo que la combinación es inmediata o nula ($O(1)$), ya que los elementos quedan ordenados *in-situ*.
- **Preservación de invariantes:** Durante la fusión, el algoritmo debe sostener las propiedades formales requeridas (por ejemplo, monotonicidad, optimalidad local o conservación de datos) para que la respuesta agregada sea matemáticamente exacta.
- **Ecuación de recurrencia global:** El costo total del paradigma queda formalizado por:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + D(n) + C(n)$$

Donde $C(n)$ representa el tiempo requerido para fusionar o ensamblar las a soluciones parciales.

8. Recurrencias

Cuando analizamos la eficiencia de los algoritmos que utilizan la estrategia de Divide y Vencerás, no podemos simplemente contar las operaciones iterativas como lo haríamos en un ciclo normal. Dado que estos

algoritmos tienen una naturaleza recursiva (se llaman a sí mismos), necesitamos una herramienta matemática que refleje este comportamiento: las ecuaciones de recurrencia.

Una recurrencia nos permite expresar el tiempo de ejecución de un problema de tamaño n en función del tiempo que toma resolver los subproblemas más pequeños. En general, el comportamiento de un algoritmo "Divide y Vencerás" se representa con la siguiente fórmula estándar:

$$T(n) = aT(n/b) + f(n)$$

Donde cada término representa un componente estructural del algoritmo:

- $T(n)$: Es el tiempo total que tarda el algoritmo para una entrada de tamaño n .
- a : Representa la cantidad de subproblemas en los que se divide el problema original ($a \geq 1$).
- n/b : Es el tamaño de cada subproblema derivado ($b > 1$).
- $f(n)$: Es el costo computacional invertido en las etapas de división ($D(n)$) y combinación ($C(n)$).

9. Teorema Maestro

Resolver las ecuaciones de recurrencia mediante métodos algebraicos (como el método del árbol de recursión o por sustitución) puede resultar complejo. Para simplificar este análisis, se utiliza el **Teorema Maestro**, el cual proporciona un método directo para determinar la complejidad asintótica en notación Big-O de recurrencias de la forma $T(n) = aT(n/b) + f(n)$.

El Teorema Maestro evalúa la relación dominante entre dos fuerzas dentro del algoritmo:

1. El trabajo acumulado en las hojas de la recursión: $O(n^{\log_b a})$.
2. El trabajo realizado en la división y combinación: $f(n)$.

Dependiendo de cuál de estos términos determine el tiempo de ejecución, se presentan tres casos:

- **Caso 1 (El trabajo en las hojas domina):** Si resolver los subproblemas consume mayor tiempo asintótico que dividir y combinar, la complejidad está determinada por la cantidad de subproblemas.

Matemáticamente: Si $f(n)$ es menor que $n^{\log_b a}$, entonces:

$$T(n) = O(n^{\log_b a})$$

- **Caso 2 (Trabajo equilibrado):** Si el costo de los subproblemas y el costo de dividir/combinar son asintóticamente equivalentes, el trabajo se distribuye de manera uniforme en cada nivel de la pila recursiva.

Matemáticamente: Si $f(n)$ es igual a $n^{\log_b a}$, entonces:

$$T(n) = O(n^{\log_b a} \log n)$$

- **Caso 3 (El trabajo en la raíz domina):** Si el tiempo invertido en la división y combinación es significativamente superior al de resolver los subproblemas, la etapa no recursiva dicta la complejidad final.

Matemáticamente: Si $f(n)$ es mayor que $n^{\log_b a}$, entonces:

$$T(n) = O(f(n))$$

10. Algoritmos representativos

10.1. Búsqueda Binaria (Binary Search)

La Búsqueda Binaria es la aplicación directa del paradigma Divide y Vencerás sobre conjuntos de datos organizados de forma secuencial y previamente ordenados. Su objetivo es localizar un elemento específico reduciendo el espacio de búsqueda en cada paso mediante la siguiente estructura lógica:

10.1.1. Fase de División (Partición del Dominio)

El algoritmo no procesa el conjunto de datos de forma lineal. En su lugar, toma el rango activo delimitado por sus extremos (*inicio* y *fin*) y calcula el punto medio exacto del espacio de búsqueda. De esta forma, el problema original de tamaño n se divide instantáneamente en dos subproblemas de tamaño aproximado $n/2$.

10.1.2. Fase de Vencimiento (Procesamiento y Caso Base)

En esta etapa, el algoritmo evalúa la posición central obtenida:

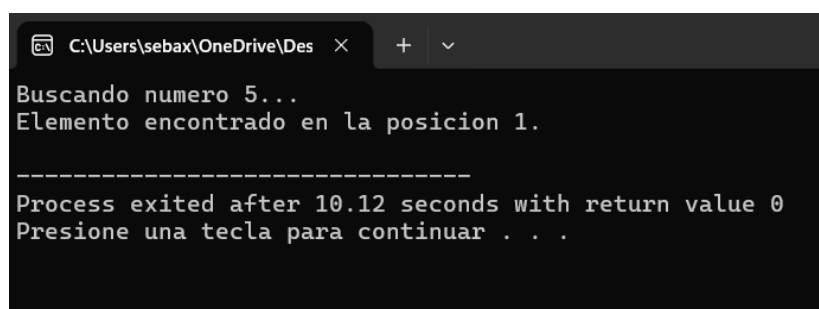
- **Caso Base (Éxito):** Si el valor ubicado en el punto medio coincide exactamente con el valor buscado, el algoritmo finaliza la búsqueda y devuelve la posición del elemento.
- **Caso Base (Fallo):** Si los límites se cruzan (*inicio* supera a *fin*), indica que el elemento no se encuentra presente en el arreglo, notificando la ausencia del valor.
- **Paso Recursivo:** Si no se alcanza ningún caso base, se compara el valor central con el objetivo:
 - Si el valor central es mayor que el buscado, se descarta la mitad derecha y la búsqueda continúa en la sublista izquierda.
 - Si el valor central es menor que el buscado, se descarta la mitad izquierda y se evalúa la sublista derecha.

10.1.3. Fase de Combinación (Integración)

En la Búsqueda Binaria la fase de combinación es inmediata. Al encontrar el elemento objetivo, la respuesta se transmite directamente a través de la pila de llamadas recursivas sin requerir procesamiento auxiliar.

10.1.4. Impacto en la Eficiencia

Gracias a la reducción continua del dominio de búsqueda ($n, n/2, n/4, n/8, \dots, 1$), el algoritmo garantiza una complejidad temporal de $O(\log n)$, permitiendo buscar elementos en grandes volúmenes de información en un número reducido de operaciones.



```
C:\Users\sebox\OneDrive\Des > Buscando numero 5...
Elemento encontrado en la posicion 1.

-----
Process exited after 10.12 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 1: Captura de ejecución de la Búsqueda Binaria localizando un elemento en el arreglo.

10.2. Ordenamiento por Fusión (Merge Sort)

Diseñado por John von Neumann en 1945, Merge Sort constituye una implementación clásica del paradigma Divide y Vencerás. Su propósito es transformar una secuencia desordenada de n elementos en una secuencia strictly ordenada.

10.2.1. Aplicación del paradigma Divide y Vencerás

A. Fase de División (Divide)

La secuencia delimitada por los índices inicio y fin se divide en dos mitades calculando el índice medio:

$$\text{medio} = (\text{inicio} + \text{fin}) / 2$$

Generando dos subarreglos de tamaño $n/2$: $A[\text{inicio} \dots \text{medio}]$ y $A[\text{medio} + 1 \dots \text{fin}]$. El costo de esta etapa es constante: $D(n) = O(1)$.

B. Fase de Conquista (Conquer)

Se resuelven los subproblemas recursivamente:

- *Caso Base*: Si el subarreglo contiene un solo elemento ($\text{inicio} = \text{fin}$) o está vacío, ya se encuentra ordenado: $T(1) = O(1)$.
- *Paso Recursivo*: Se ejecutan dos llamadas independientes sobre cada mitad:
`merge_sort(A, inicio, medio);`
`merge_sort(A, medio + 1, fin);`

C. Fase de Combinación (Combine)

Se ejecuta la subrutina auxiliar merge:

- *Mecanismo de fusión*: Se comparan los elementos de ambos subarreglos ya ordenados y se intercalan de manera secuencial en un arreglo temporal.
- *Costo computacional*: Recorrer los elementos requiere tiempo lineal proporcional al número de elementos: $C(n) = O(n)$.

10.2.2. Formulación y análisis de complejidad temporal

La relación de recurrencia de Merge Sort se expresa como:

$$\text{Si } T(n) = O(1), \text{ si } n = 1$$

$$\text{Si } T(n) = 2 \cdot T(n/2) + O(n), \text{ si } n > 1$$

10.2.3. Resolución mediante el Teorema Maestro

Comparando con la forma general $T(n) = a * T(n / b) + O(n^d)$:

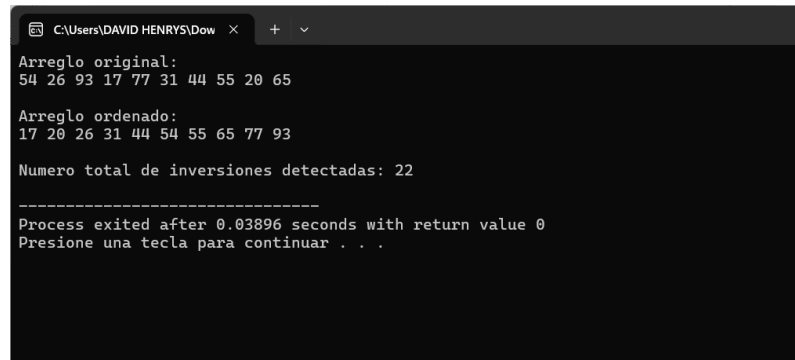
- $a = 2, b = 2 \implies \log_b(a) = \log_2(2) = 1$.
- $d = 1$.

Dado que $d = \log_b(a) = 1$, aplica el **Caso 2 del Teorema Maestro**:

$$T(n) = O(n \log n)$$

A diferencia de otros algoritmos, Merge Sort mantiene un rendimiento determinista de $O(n \log n)$ tanto en el

peor, promedio como en el mejor caso.



```
C:\Users\DAVID HENRYS\Down >
Arreglo original:
54 26 93 17 77 31 44 55 20 65

Arreglo ordenado:
17 20 26 31 44 54 55 65 77 93

Numero total de inversiones detectadas: 22

-----
Process exited after 0.03896 seconds with return value 0
Presione una tecla para continuar . . .
```

Figura 2: Ejecución de Merge Sort ordenando una secuencia de enteros y calculando inversiones.

10.2.4. Evaluación de propiedades operativas

- **Estabilidad algorítmica:** Es un algoritmo estable. Al fusionar dos elementos iguales ($A[i] = A[j]$), se prioriza el elemento del subarreglo izquierdo, conservando el orden relativo original.
- **Consumo de memoria auxiliar:** Posee una complejidad espacial de $O(n)$ debido a la necesidad de emplear un arreglo temporal durante la fase de combinación.
- **Patrón de acceso a memoria:** Su acceso secuencial beneficia el uso de memoria caché y lo hace adecuado para el ordenamiento de grandes volúmenes de datos en almacenamiento secundario o listas enlazadas.

10.3. Ordenamiento Rápido (Quick Sort)

Quick Sort es un algoritmo de ordenamiento eficiente basado en Divide y Vencerás que enfoca la mayor parte del trabajo computacional en la fase de división mediante la selección de un pivote.

10.3.1. Aplicación del paradigma Divide y Vencerás

- **A. Fase de División (Divide):** Se selecciona un elemento del arreglo denominado *pivote*. El arreglo se reorganiza (particiona) de modo que los elementos menores o iguales al pivote se ubican a su izquierda y los mayores a su derecha. El costo de esta partición es lineal: $D(n) = O(n)$.
- **B. Fase de Conquista (Conquer):** Se ordenan recursivamente las dos sublistas generadas a ambos lados del pivote. Caso base: subarreglos de tamaño 0 o 1 ($T(1) = O(1)$).
- **C. Fase de Combinación (Combine):** Como la reorganización de elementos se realiza directamente sobre el arreglo original (*in-place*), la fase de combinación es nula: $C(n) = O(1)$.

10.3.2. Formulación y análisis de complejidad temporal

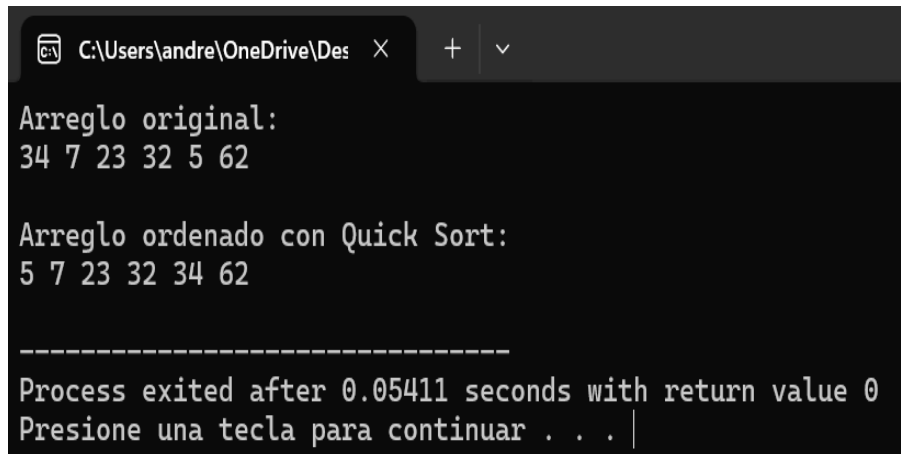
En el caso promedio (asumiendo particiones equilibradas de tamaño $n / 2$), la recurrencia es:

$$T(n) = 2 * T(n / 2) + O(n)$$

Aplicando el Caso 2 del Teorema Maestro, se obtiene una complejidad promedio de $O(n \log n)$. No obstante, si el pivote elegido es desfavorable (por ejemplo, el menor o mayor elemento en un arreglo ya ordenado), el árbol se desequilibra y la complejidad puede degradarse a $O(n^2)$.

10.3.3. Evaluación de propiedades operativas

- **Consumo de memoria auxiliar:** Presenta una complejidad espacial de $O(\log n)$ correspondiente a la profundidad de la pila de llamadas recursivas, operando sin requerir arreglos auxiliares (*in-place*).
- **Estabilidad y rendimiento:** No es un algoritmo estable por defecto. Su velocidad práctica es elevada, aunque requiere estrategias adecuadas de selección de pivote (pivote aleatorio o mediana de tres) para prevenir el peor caso $O(n^2)$.



```
C:\Users\andre\OneDrive\Des X + v

Arreglo original:
34 7 23 32 5 62

Arreglo ordenado con Quick Sort:
5 7 23 32 34 62

-----

Process exited after 0.05411 seconds with return value 0
Presione una tecla para continuar . . . |
```

Figura 3: Ejecución de Quick Sort ordenando un conjunto de números enteros.

10.4. Tabla Comparativa General

Algoritmo	Complejidad Temporal Promedio	Complejidad Peor Caso	Complejidad Espacial	Ventaja Principal	Desventaja Principal
Búsqueda Binaria	$O(\log n)$	$O(\log n)$	$O(\log n)$	Búsqueda ultra rápida en grandes volúmenes.	Exige que el arreglo esté previamente ordenado.
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Estable y rendimiento garantizado $O(n \log n)$.	Consume memoria RAM extra ($O(n)$).
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Muy rápido en la práctica; ordena <i>in-place</i> .	Sensible a la elección del pivote (peor caso $O(n^2)$).

11. Conclusiones

El análisis del paradigma Divide y Vencerás demuestra que la optimización algorítmica no depende exclusivamente de la capacidad de procesamiento del hardware, sino de la estructuración matemática del problema subyacente. Herramientas analíticas como el Teorema Maestro prueban que es posible modelar y predecir con exactitud la complejidad asintótica de estas implementaciones recursivas antes de su ejecución. Asimismo, la exploración de métodos avanzados como el algoritmo de Karatsuba evidencia que la verdadera potencia de esta estrategia no solo está en dividir los datos, sino en la capacidad de reestructurar algebraicamente un problema para disminuir la cantidad de llamadas recursivas en la fase de conquista; logrando así quebrar las barreras de los enfoques tradicionales de costo cuadrático para procesar volúmenes masivos de información con una eficiencia superior.

— *Max André Alegre Bustamante*

El estudio del paradigma Divide y Vencerás (*Divide and Conquer*) permitió comprender cómo la estrategia de tres pasos —Dividir, Vencer (mediante recursión) y Combinar— optimiza el rendimiento computacional al resolver subproblemas cada vez más pequeños hasta un caso base trivial. Como resultado práctico, la prueba de la Búsqueda Binaria demostró que es posible reducir la complejidad temporal de $O(n)$ a $O(\log n)$, logrando descartar la mitad de los datos en cada paso y procesando grandes volúmenes de información en muy pocas operaciones.

— *Sebastian Matias Huatuco Rojas*

El paradigma Divide y Vencerás se consolida como uno de los pilares más determinantes de la computación gracias a su capacidad de transformar problemas intratables o de costo cuadrático en soluciones óptimas de orden logarítmico, operando bajo una división simétrica de subproblemas independientes resueltos por recursión y una posterior combinación eficiente. A través de este trabajo se evidenció no solo la rigurosidad teórica de sus tres fases analizadas formalmente mediante relaciones de recurrencia, sino también su versatilidad práctica plasmada en Merge Sort; algoritmo que trasciende el ordenamiento convencional garantizando una cota determinista estricta de $O(n \log n)$ y sirviendo de base analítica para resolver retos avanzados como el conteo de inversiones globales en telemetría en tiempo óptimo, demostrando que la mayor ganancia algorítmica de esta técnica radica en sintetizar y extraer información durante la etapa de combinación sin incurrir en búsquedas exhaustivas.

— *Flores Apaza Joel Josue*

12. Bibliografía

- Frias, S. (2021). *Significado del algoritmo divide y vencerás: Explicado con ejemplos*. freeCodeCamp. Recuperado de: <https://www.freecodecamp.org/espanol/news/significado-del-algoritmo-divide-y-venceras/>
- Coddy. (2026, 12 julio). *Búsqueda binaria*. Coddy. <https://coddy.tech/visualize/es/searching/binary-search>
- Programiz. (s. f.). *Binary Search (With Code)*. <https://www.programiz.com/dsa/binary-search>
- GeeksforGeeks. (2026, 26 agosto). *Introduction to Divide and Conquer Algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/introduction-to-divide-and-conquer-algorithm/>
- Chio Code. (2021, 3 marzo). *Aprende divide y vencerás | Subarreglo máximo | Diseño de algoritmos* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=UxtAqHOb8aw>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press. <https://doi.org/10.7551/mitpress/12513.001.0001>
- Kleinberg, J., & Tardos, É. (2014). *Algorithm design*. Pearson. <https://www.pearson.com/en-us/subject-catalog/p/algorithm-design/P2000000003534>
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley Professional. <https://algs4.cs.princeton.edu/home/>
- Knuth, D. E. (1998). *The art of computer programming: Sorting and searching* (Vol. 3, 2.^a ed.). Addison-Wesley. <https://www-cs-faculty.stanford.edu/~knuth/taocp.html>
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2.^a ed.). Prentice Hall. <https://www.bell-labs.com/usr/dmr/www/cbook.html>